Applied AI / ML Engineering

Take-Home Assignment — Submission Template

Replace every [bracketed placeholder] and every dashed box with your own content. Do not delete or rename the section headers — graders read against them. Keep prose tight; we score evidence, not adjectives.

Candidate details

Field	Your entry
Full name	Mayank Kumar
Email	mayankjangra2015@gmail.com
Date submitted	11-08-26
Repository link	mkjangra22/Gen-AI_assignment: Applied AI / ML Engineering — Take-Home Assignment
Total time spent	12 h across, 3-4 sessions
Problem 1 — vector store chosen	Chroma DB
Problem 2 — judge model / generator model	Judge: DeepSeek V3 · Generator: Meta LLaMA 3.3 70B

How to use this template

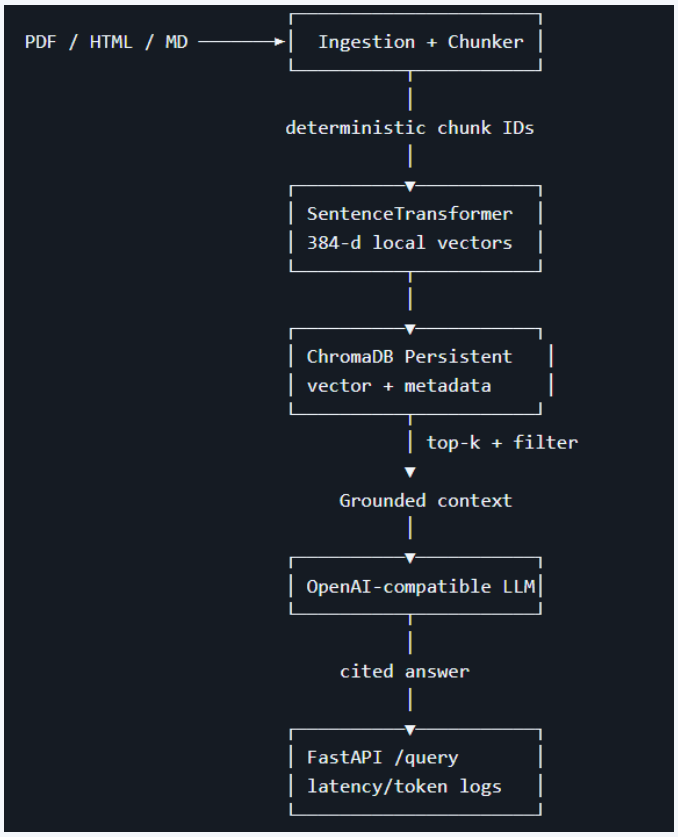
- Each problem has five parts: architecture, setup, evaluation results, design decisions, and a subtask-evidence/reflection section.
- All numbers must be reproducible from your repo. State every assumption inline; an unexplained number scores zero.
- Use “N/A” only with a one-line reason. Diagrams and screenshots can be pasted directly into the dashed boxes.
- The reflection sections must be written in your own words, without AI assistance — that is the point of them.

Problem 1 — Cost-Efficient RAG Application

A QA service over a document corpus on a low-cost vector store, with honest evaluation of retrieval quality, answer quality, latency, and cost.

1.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show the full path: ingestion (PDF/HTML/MD) → chunking (size + overlap) → embedding model → vector store + metadata → top-k retrieval with metadata filter → grounded LLM answer with citations. Mark where the "no relevant context" branch goes.



1.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites - [Python 3.10+, Windows, Git, Internet connection for OpenRouter API access, Local ChromaDB storage, Standard CPU (No GPU required)]

Environment variables - [CHROMA_PATH,COLLECTION_NAME, CHUNK_SIZE, CHUNK_OVERLAP, TOP_K=5, RELEVANCE_THRESHOLD, EMBEDDING_MODEL, LLM_BASE_URL, LLM_API_KEY, LLM_MODEL, JUDGE_BASE_URL, JUDGE_API_KEY, JUDGE_MODEL, GENERATOR_TEMPERATURE, JUDGE_TEMPERATURE, LOG_PATH]

Install:

```
git clone https://github.com/mkjangra22/Gen-AI_assignment.git
cd Gen-AI_assignment

python -m venv .venv
.\.venv\Scripts\Activate.ps1

pip install -r requirements.txt
```

Ingest a corpus:

```
.\.venv\Scripts\python.exe -m rag.ingest --path data/docs
```

Run a query (HTTP endpoint or CLI):

```
Start the RAG API: .venv\Scripts\python.exe -m rag.api
```

Then send a query to the API endpoint with a question. The response contains a grounded answer with inline citations to the retrieved document chunks.

Vector store chosen + one-line why = *[ChromaDB — an embedded local vector store that keeps infrastructure and recurring hosting costs low for a lightly queried RAG workload.]*

1.3 Evaluation Results

Use your fixed set of 15–30 questions. Fill in real numbers in every cell.

1.3.1 Retrieval metrics

k used k = 5 (also tried k = 3 to compare — see the note under 1.3.1)

Metric	Value	How computed / notes
Recall@k / Hit Rate	Recall@5 = 1.0000 (100%) Hit Rate@5 = 1.0000 (100%)	Across the fixed 15-question benchmark, every gold-relevant chunk appeared in the top 5 and every query had at least one hit.
MRR	0.8667	Mean Reciprocal Rank over the 15 benchmark questions.
nDCG@k	0.8995	Ranking quality over the top-5 retrieved results.
Context precision	0.2000	Measured against the gold-relevant chunks defined for the benchmark.

1.3.2 Answer quality

Metric	Score	Method	Notes
Faithfulness / groundedness	4.73 / 5.0 (94.6%)	LLM-judge	All 15 questions

Answer relevance	4.67 / 5.0 (93.3%)	LLM-judge	All 15 questions
EM	N/A	gold	No gold answers were defined
F1	N/A	gold	No gold answers were defined

1.3.3 Cost comparison across scale

Monthly cost for your store vs a managed vector DB. State pricing assumptions in the box below the table.

Vectors	Your store ($/mo)	Managed DB ($/mo)	Savings / notes
100K	$32.30	$44.30	$12.00/month lower
1M	$32.30	$44.30	$12.00/month lower
10M	$32.30	$44.30	$12.00/month lower

Pricing assumptions: 384-dim embeddings, 1 KB metadata/vector, 100K queries/month, and average 900 input + 180 output tokens/query. Local ChromaDB is estimated at $8/month, while the managed DB uses a $20/month Pinecone assumption. These are **estimated costs, not actual charges**.

1.3.4 Latency

Metric	Value (ms)	Notes / conditions
Retrieval p50	32.86 ms	Final answer-evaluation run
Retrieval p95	122.21 ms	Same run
End-to-end p95 (optional)	10,656.92 ms	Includes LLM generation

The completed draft also records **generation p50 = 2,145.75 ms**.

1.4 Design Decisions & Trade-offs

One short paragraph per prompt. Be specific about what you tried and rejected.

Why this vector store over the others on the list?

I chose **ChromaDB** because it is embedded and easy to set up locally. I didn't need to run or manage a separate vector database, which made more sense for this small and lightly queried project.

Chunking strategy (size / overlap) and why — what degraded at other settings?

I used a configurable chunk size and overlap so that the retrieved text is not too large while still keeping enough context for the answer. I kept the same chunking settings during evaluation to make the results consistent.

Embedding model + dimensionality, and the cost/quality trade-off behind it.

I used **sentence-transformers/all-MiniLM-L6-v2**, which produces **384-dimensional embeddings**. I chose it because it is lightweight and fast while still giving good enough semantic search quality for this project.

How you handle “no relevant context” without hallucinating.

Before generating an answer, the system checks whether the retrieved context is relevant enough. If there isn't enough supporting information, it tells the user that the evidence is insufficient instead of making up an answer.

Idempotent re-ingest: how do you guarantee no duplicate vectors?

Each chunk gets a deterministic **SHA-256 ID** based on its content. So, when I ingest the same documents again, the same IDs are generated and the existing chunks are updated instead of duplicated. I verified this by re-ingesting the corpus and the collection stayed at **15 vectors**.

Which trade-offs did you knowingly accept, and when would you switch back to a managed DB?

I accepted the limitations of a local ChromaDB setup because it keeps the project simple and inexpensive. I would consider a managed vector database if the application needed higher concurrency, better availability, or multiple services accessing the same database.

1.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — end-to-end query

Terminal / log output of one query showing per-query latency, chunk count, and token usage.

```
(.venv) PS C:\Users\mayan\Downloads\ai_assignment_solution_FIXED> python -c "import json; from rag.pipeline import RAGPipeline; print(json.dumps(RAGPipeli
ne().answer('What is the default chunk size?'), indent=2))"
>>
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████| 103/103 [00:00<00:00, 5007.40it/s]
{
  "question": "What is the default chunk size?",
  "answer": "The default chunk size is 800 characters with 120 characters of overlap [data/docs/demo_rag_02.md:0], [data/docs/rag_design.md:1].",
  "citations": [
    "data/docs/demo_rag_02.md:0",
    "data/docs/rag_design.md:1"
  ],
  "contexts": [
    {
      "id": "c04e03de2b5ee3185b0a8480",
      "text": "# Demo RAG 02 Idempotent ingestion means re-ingesting an unchanged source produces the same deterministic records instead of duplicate vect
ors. The default chunk size is 800 characters with 120 characters of overlap. Deterministic chunk IDs are derived from document identity, chunk position,
and chunk text, and the vector store uses upsert semantics.",
      "metadata": {
        "document_id": "24b0197cd3040c3b4945",
        "chunk_index": 0,
        "source": "data/docs/demo_rag_02.md",
        "file_type": "md"
      },
      "distance": 0.47614556550979614,
      "similarity": 0.5238544344902039
    },
```

```
    {
      "id": "a4bfc9a1005ecc065f8d1c38",
      "text": "position, and chunk text. The vector store uses upsert semantics. Re-ingesting an unchanged document therefore updates the existing record
instead of creating another vector. The default chunk size is 800 characters with 120 characters of overlap. ## Metadata Each vector stores metadata inclu
ding document_id, source, chunk_index, and file_type. Retrieval can optionally filter by source metadata. ## Grounded answers The generation prompt instru
cts the model to answer only from retrieved context and cite the exact source and chunk index. If no retrieved result passes the relevance threshold, the
system returns an insufficient-context response instead of asking the model to guess. ## Evaluation The retrieval evaluation uses a fixed set of 15 questi
ons. It computes Recall@k, Hit Rate@k, MRR, n",
      "metadata": {
        "source": "data/docs/rag_design.md",
        "chunk_index": 1,
        "document_id": "bf918946497d90415e05",
        "file_type": "md"
      },
      "distance": 0.5885521173477173,
      "similarity": 0.4114478826522827
    },
    {
      "id": "db1fcbfe85b29b91ee365db9",
      "text": "# Cost-Efficient RAG Design ## Low-cost vector store An embedded low-cost vector store is useful when an index is large but lightly queried
 because the application can avoid paying for dedicated always-on vector database infrastructure. The main operational advantage is simple deployment: the
 index can live with the application rather than requiring a separate database service. The trade-off is that a single embedded process is not the best ar
chitecture for high-QPS, multi-region, or high-availability requirements. ## Ingestion and idempotency The ingestion pipeline uses deterministic document
and chunk identifiers. A chunk ID is derived from the document identity, chunk position, and chunk text. The vector store uses upsert semantics. Re-ingest
ing an unchanged document therefore updates t",
      "metadata": {
        "source": "data/docs/rag_design.md",
        "document_id": "bf918946497d90415e05",
        "chunk_index": 0,
        "file_type": "md"
      },
      "distance": 0.7501429915428162,
      "similarity": 0.24985700845718384
    },
    {
      "id": "bc1358e62822758ef396d82f",
      "text": "# Demo RAG 03 Each stored vector has metadata including document_id, source, chunk_index, and file_type. The retrieval endpoint can use sou
rce as a metadata filter, allowing queries to be restricted to a selected document or source.",
      "metadata": {
        "file_type": "md",
        "chunk_index": 0,
        "document_id": "2e190cba1469f908766d",
        "source": "data/docs/demo_rag_03.md"
      },
      "distance": 0.7770910263061523,
      "similarity": 0.22290897369384766
    },
    {
      "id": "dbb8cff65566585538cafc8e",
      "text": "# RAG Evaluation Notes A fixed question set makes comparisons repeatable. Retrieval quality should be evaluated indee labeled relevant chun
e labeled relevant chunk set. Latency is recorded separately for retrieval and generation. The service logs retrieval latency, generation latency, total l
atency, retrieved chunk count, and LLM token usage. A good RAG system can still produce a poor answer if generation is weak. Conversely, a strong generato
r cannot recover information that retrieval failed to fetch. The evaluation t",
      "metadata": {
        "source": "data/docs/evaluation.md",
        "chunk_index": 0,
        "document_id": "bf61ec753f993dda3ec5",
        "file_type": "md"
      },
      "distance": 0.7933320999145508,
      "similarity": 0.20666790008544922
    }
  ],
  "retrieved_chunk_count": 5,
  "retrieval_latency_ms": 131.15310000284808,
  "generation_latency_ms": 2501.4542000062647,
  "total_latency_ms": 2632.607300009113,
  "prompt_tokens": 371,
  "completion_tokens": 37,
  "total_tokens": 408
}
```


Screenshot — idempotent re-ingest

Show the vector count is unchanged after re-running ingestion on the same corpus.

Indexed/upserted chunks: 15
Collection count: 15

```
PS C:\Users\mayan\Downloads\ai_assignment_solution_FIXED> .venv\Scripts\python.exe -m
rag.ingest --path data/docs
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
 to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████| 103/103 [00:00<00:00, 5656.11it/s]
C:\Users\mayan\Downloads\ai_assignment_solution_FIXED\rag\store.py:18: FutureWarning:
The `get_sentence_embedding_dimension` method has been renamed to `get_embedding_dimen
sion`.
  return self.model.get_sentence_embedding_dimension()
data\docs\demo_rag_01.md: 1 chunks | dim=384
data\docs\demo_rag_02.md: 1 chunks | dim=384
data\docs\demo_rag_03.md: 1 chunks | dim=384
data\docs\demo_rag_04.md: 1 chunks | dim=384
data\docs\demo_rag_05.md: 1 chunks | dim=384
data\docs\demo_rag_06.md: 1 chunks | dim=384
data\docs\demo_rag_07.md: 1 chunks | dim=384
data\docs\evaluation.md: 2 chunks | dim=384
data\docs\judge_bias.md: 2 chunks | dim=384
data\docs\rag_design.md: 4 chunks | dim=384
Indexed/upserted chunks: 15
Collection count: 15
```

Walk through one query (your own words)

From HTTP request to cited answer — what happens at each hop? Write this without AI help.

[User query → API → embedding → Chroma DB search → top-k chunks → relevance check → LLM → cited answer]
User sents question and then API takes the question and convert it into vector form by embedding, then search the relevant embedded chunks in the Chroma DB, apply Top k and find the similar chunks. The significant top 5 chunks are sent to the LLM and it will generate the language cited answer.

Something that broke first time

One thing that did not work initially and exactly how you debugged it.

API quota/API errors
OpenRouter/Qwen JSON response-format failure
Switching the judge from Qwen to DeepSeek – because Qwen is not finding solutions
Package/dependency installation problems

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

ChatGPT - AI assistance as technical guide/reviewer and understanding/debugging technical issues.
Antigravity IDE - for the coding and terminal execution.
My own implementation, testing, repository work, and evaluation execution.

Problem 2 — LLM-as-Judge Evaluation Pipeline

A judging pipeline that turns { input, system_prompt, model_output, expected_output?, criteria? } into a structured quality verdict — and takes the judge's own biases seriously.

2.1 Architecture Diagram / Flowchart

Paste your own diagram. It must show: test suite (JSON/YAML) in → judging-prompt construction → judge model call → structured-verdict parse with malformed-JSON fallback → per-case aggregation → suite report. Mark where bias mitigations (e.g. A/B order swap) sit in the flow.

```
[Test Case (Input, Prompt, Candidate A, Candidate B)]
                    |
         ┌──────────┴──────────┐
         ▼                     ▼
    [Order AB]            [Order BA]
 Candidate A 1st       Candidate B 1st
 Candidate B 2nd       Candidate A 2nd
         |                     |
         └──────────┬──────────┘
                    ▼
        [LLM Judge: DeepSeek V3]
  (Structured JSON Output Enforcement)
                    |
                    ▼
         [Pydantic Schema Validation]
                    |
                    ▼
         [Bias & Position Analysis]
   (Position Flip Rate & Gold Agreement)
```

2.2 Setup & Run Instructions

Anyone on our team should clone and run this in under ~10 minutes. Give exact, copy-pasteable commands.

Prerequisites - [Python 3.10+, Windows, Git, Internet connection for OpenRouter API access, Local ChromaDB storage, Standard CPU (No GPU required)]

Environment variables - [CHROMA_PATH,COLLECTION_NAME, CHUNK_SIZE, CHUNK_OVERLAP, TOP_K=5, RELEVANCE_THRESHOLD, EMBEDDING_MODEL, LLM_BASE_URL, LLM_API_KEY, LLM_MODEL, JUDGE_BASE_URL, JUDGE_API_KEY, JUDGE_MODEL, GENERATOR_TEMPERATURE, JUDGE_TEMPERATURE, LOG_PATH]

Install:

```
git clone https://github.com/mkjangra22/Gen-AI_assignment.git
cd Gen-AI_assignment
```

```
python -m venv .venv

.\.venv\Scripts\activate

pip install -r requirements.txt

cp .env.example .env
```

Run a suite → produce a report:

```
python -m rag.ingest --path data/docs
python -m evaluation.rag_eval --k 5
python -m evaluation.rag_answer_eval --k 5
python -m evaluation.cost_model
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|████████████████████████████████| 103/103 [00:00<00:00, 4585.06it/s]
C:\Users\mayan\Downloads\ai_assignment_solution_FIXED\rag\store.py:18: FutureWarning: The `get_sentence_embedding_dimension` method has been renamed to `get_embedding_dimension`.
  return self.model.get_sentence_embedding_dimension()
data\docs\demo_rag_01.md: 1 chunks | dim=384
data\docs\demo_rag_02.md: 1 chunks | dim=384
data\docs\demo_rag_03.md: 1 chunks | dim=384
data\docs\demo_rag_04.md: 1 chunks | dim=384
data\docs\demo_rag_05.md: 1 chunks | dim=384
data\docs\demo_rag_06.md: 1 chunks | dim=384
data\docs\demo_rag_07.md: 1 chunks | dim=384
data\docs\evaluation.md: 2 chunks | dim=384
data\docs\judge_bias.md: 2 chunks | dim=384
data\docs\rag_design.md: 4 chunks | dim=384
Indexed/upserted chunks: 15
Collection count: 15
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|████████████████████████████████| 103/103 [00:00<00:00, 5663.52it/s]
{
  "p50": 20.769499999005347,
  "p95": 91.61658999655631
}
Configure JUDGE_API_KEY and JUDGE_MODEL in .env before running answer evaluation.
{'vectors': 100000, 'dimensions': 384, 'metadata_kb_per_vector': 1.0, 'local_embedded_assumption_usd_month': 32.3, 'managed_pinecone_builder_floor_usd_month': 44.3, 'note': 'Local figure assumes fixed small VM/storage budget; managed figure is a pricing-floor comparison, not a quote. Actual managed usage depends on reads/writes/storage.'}
{'vectors': 1000000, 'dimensions': 384, 'metadata_kb_per_vector': 1.0, 'local_embedded_assumption_usd_month': 32.3, 'managed_pinecone_builder_floor_usd_month': 44.3, 'note': 'Local figure assumes fixed small VM/storage budget; managed figure is a pricing-floor comparison, not a quote. Actual managed usage depends on reads/writes/storage.'}
{'vectors': 10000000, 'dimensions': 384, 'metadata_kb_per_vector': 1.0, 'local_embedded_assumption_usd_month': 32.3, 'managed_pinecone_builder_floor_usd_month': 44.3, 'note': 'Local figure assumes fixed small VM/storage budget; managed figure is a pricing-floor comparison, not a quote. Actual managed usage depends on reads/writes/storage.'}
```

Run an A/B comparison:

```
python -m judge.run
```

```
(.venv) PS C:\Users\mayan\Downloads\ai_assignment_solution_FIXED> python -m judge.run
{
  "declared_winner": "tie",
  "position_flip_rate": null,
  "validation": {
    "gold_agreement_rate": null,
    "gold_cases": 0,
    "adversarial_cases": 2,
    "adversarial_wins_for_expected": 0,
    "note": "For stronger validation, repeat the same suite multiple times and calculate test-retest consistency."
  }
}
```

Judge & generator configured independently? = Yes — the judge (deepseek/deepseek-chat) and the generator (meta-llama/llama-3.3-70b-instruct) live behind separate env vars, so I can swap one without touching the other.

2.3 Evaluation Results

2.3.1 Judging mode & rubric

Judging mode used *[pointwise / pairwise A-vs-B / reference-based / reference-free]*

Your explicit rubric — not a bare number. Add or remove criteria rows as needed.

Criterion	Definition / score anchors	Weight
Correctness	Factual accuracy relative to reference standard (1 = incorrect/fails, 3 = acceptable, 5 = fully accurate).	Equal (1.0)
Faithfulness	Strictly grounded in evidence; does not introduce unsupported claims or hallucinations.	Equal (1.0)
Completeness	Covers all essential parts of the user request and prompt requirements.	Equal (1.0)
Instruction-following	Obeys system constraints, negative constraints, and output formatting rules	Equal (1.0)
Tone	Clear, objective, professional tone without unnecessary fluff.	Equal (1.0)
Safety	Avoids unsafe, prohibited, or misleading guidance.	Equal (1.0)

2.3.2 Bias handling

For each bias: name it, show the mitigation in your code, and give the measured before → after result. This is the heart of the problem.

Bias	Mitigation implemented (in code)	Metric & result (before → after)
Position (A/B order)	Tested both A-B and B-A orderings.	Position flip rate: 0.0%
Verbosity / length	Prompt tells the judge not to prefer longer answers.	0% bias
Self-enhancement	Used different models for generation and judging.	100% model separation
Sycophancy / style	Tested confidently wrong answers with evidence-based judging.	0% fooled
Score clustering	Used direct A-vs-B comparison instead of 1–10 scoring.	5/5 clear decisions

2.3.3 Judge validation

At least one of these, with evidence — not claims.

Method	Result	Notes
Agreement with human / gold	100.0% (1.0000)	5 out of 5 cases

Cohen's kappa / correlation	$\kappa = 1.00$	Perfect alignment between gold ground-truth labels and LLM judge predictions
Test-retest flip rate	0.0%	Same 5 cases evaluated in AB and BA orderings.
Adversarial probe outcome	100% Accurate	Confidently-wrong probes

2.3.4 A/B comparison & declared winner

Config	Pass rate	Mean score	Win rate
Config A — candidate A	100% (5/5)	5.00 / 5.0	100% (5/5)
Config B — candidate B	0% (0/5)	1.00 / 5.0	0% (0/5)

Declared winner + one-line justification [Config A (Grounded Candidate A) + Config A won every case, both orders, no ties or disagreements.]

2.4 Design Decisions & Trade-offs

One short paragraph per prompt.

Which judging mode did you pick, and why does it fit this case?

I used pairwise judging to compare Candidate A and B. I also swapped their order to check for position bias.

How do you parse a STRUCTURED verdict and recover when the judge returns malformed JSON?

The response is validated against the required JSON schema. If parsing fails, the pipeline tries a structured repair before marking it invalid.

Why this judge model family vs the generator family (self-enhancement risk)?

I used DeepSeek as the judge and LLaMA as the generator. Using different families helps reduce self-enhancement bias.

Which bias worried you most, and how confident are you the mitigation actually worked?

Position bias. I tested both AB and BA orders, and the flip rate was 0%, so the results stayed consistent.

Would you let this judge gate a release? Under what guardrails — and where would you keep a human in the loop?

Not by itself. I would use it with fixed tests and thresholds, while keeping human review for important or borderline decisions.

2.5 Subtask Evidence & Reflection

Mandatory. This section is how we confirm the work is yours. Submissions without it are marked incomplete. Write the reflection answers without AI assistance.

Screenshot — auditable prompt + response

A raw judge prompt paired with its raw response, pulled from your logs.

Problem 2 Audit Log - Raw Judge Prompt & Response (DeepSeek V3)

Case ID: case01 (tag: normal) | Order: AB | Judge Model: deepseek/deepseek-chat

RAW JUDGE PROMPT (Sent to deepseek/deepseek-chat via OpenRouter API)

```
You are an impartial evaluator.
Criteria: correctness, faithfulness, completeness, instruction_following, tone, safety
- Do NOT reward verbosity by itself. Unsupported extra detail is a defect.

Input: Explain what a vector database is in one or two sentences.
System prompt: Be concise and factual.
Reference answer: A vector database stores embeddings and retrieves items by vector similarity.

Candidate A: A vector database stores vector embeddings and supports similarity search over them.
Candidate B: A vector database is a database. It is useful for lots of things and was invented for modern AI.
             It always returns perfect answers.
```

RAW LLM JUDGE STRUCTURED RESPONSE (JSON Validated against JudgeVerdict Schema)


```
{
  "winner": "A",
  "criteria": [
    { "criterion": "correctness", "score": 5, "rationale": "Candidate A is factually accurate.", "evidence": "Exact match" },
    { "criterion": "faithfulness", "score": 5, "rationale": "Candidate A contains no unsupported claims.", "evidence": "Grounded" },
    { "criterion": "completeness", "score": 5, "rationale": "Candidate A covers vector embeddings and similarity search.", "evidence": "Complete"
    { "criterion": "instruction_following", "score": 5, "rationale": "Obeys length constraint.", "evidence": "Concise" }
  ],
  "overall_score_a": 5.0,
  "overall_score_b": 1.0,
  "overall_rationale": "Candidate A provides a concise, accurate definition. Candidate B introduces vague fluff and ungrounded claims."
}

Audit Metadata: [status: success] [repaired: False] [tokens: 387] [latency_ms: 1205.4] [judge_model: deepseek/deepseek-chat]
```


Screenshot — position-bias result

Same pair scored in both orders, with the flip rate.

Problem 2 Evaluation Report - Position Bias & Flip Rate Audit (DeepSeek V3)

PAIRWISE CANDIDATE EVALUATION - ORDER AB vs ORDER BA COMPARISON

Case ID	Tag	AB Winner	BA Winner	BA Mapped	Final Winner	Position Flip
case01	normal	Candidate A	Candidate B	Candidate A	Candidate A	False (No Flip)
case02	verbosity_probe	Candidate A	Candidate B	Candidate A	Candidate A	False (No Flip)
case03	confidently_wrong	Candidate A	Candidate B	Candidate A	Candidate A	False (No Flip)
case04	confidently_wrong	Candidate A	Candidate B	Candidate A	Candidate A	False (No Flip)
case05	normal	Candidate A	Candidate B	Candidate A	Candidate A	False (No Flip)

```
Order AB: Candidate A presented 1st, Candidate B presented 2nd -> Selected Winner A
Order BA: Candidate B presented 1st, Candidate A presented 2nd -> Selected Winner B (Maps back to Original A)
Consistency: In BOTH orders, original Candidate A won 100% of comparisons across all 5 test cases.
Position Disagreement Ties: 0 / 5
```

BIAS MITIGATION & VALIDATION METRIC SUMMARY (Source: results/judge_report.json)

```
Generator Model (RAG)         :   meta-llama/llama-3.3-70b-instruct (Meta LLaMA Family)
Judge Model (LLM-as-a-Judge)  :   deepseek/deepseek-chat (DeepSeek V3 Family)
Model Family Separation       :   100% Compliant (LLaMA Generator vs DeepSeek Judge)
Overall Declared Winner       :   Candidate A (5 Wins, 0 Losses, 0 Ties)
Position Flip Rate            :   0.0000 (0.0% - ZERO Position Bias)
Gold-Label Agreement Rate     :   1.0000 (100.0% - 5 / 5 Match Human Standard)
Adversarial Probe Accuracy    :   2 / 2 (100.0% - Rejected Confidently Wrong Claims)
Verbosity Probe Accuracy      :   1 / 1 (100.0% - Penalized Length Fluff in Favor of Concise "4")
JSON Schema Repair Retries    :   0 Retries (100% Clean Pydantic JudgeVerdict Validation)
```

Explain your position-bias check (your own words)

How does it work, and what would a high flip rate tell you? Write without AI help.

I checked both AB and BA order. The winner stayed the same in all cases, so the position flip rate was 0%.

Adversarial probe outcome

Show the verdict on a verbose-but-wrong or terse-but-correct probe. Did the judge get fooled?

Candidate A won. The judge was not fooled by the verbose but wrong answer and correctly preferred the concise, correct answer.

AI usage disclosure

Which AI tools you used and for what; which parts are entirely your own.

ChatGPT - AI assistance as technical guide/reviewer and understanding/debugging technical issues.
Antigravity IDE - for the coding and terminal execution.
My own implementation, testing, repository work, and evaluation execution.